

Google Antigravity Customization Architecture

Implementing Project-Scoped Safety & Code Formatting Rules via .agents/AGENTS.md

Overview: In Google Antigravity, `AGENTS.md` (and `GEMINI.md`) files establish **ambient operational guardrails**. Unlike procedural skills that require event matching, rules in `.agents/AGENTS.md` are automatically discovered by hierarchical upward traversal and enforced continuously across all agent turns. They are the primary mechanism for establishing safety boundaries, formatting standards, and verification mandates.

1. Core Mechanics of AGENTS.md Rule Discovery

Antigravity enforces rules using a deterministic hierarchical directory model:

Mechanism	Operational Behavior	Architectural Advantage
Upward Traversal	The agent walks from the active file / CWD up to repository root, loading all discovered <code>AGENTS.md</code> / <code>GEMINI.md</code> files.	Enables root-level global policies with fine-grained subsystem overrides.
Pure Markdown Format	No YAML frontmatter required. Plain Markdown with headers, bullet points, and code fences.	Simple to maintain, read, and review in Git pull requests.
Automatic Deduplication	Rules are deduplicated by resolved canonical file path.	Prevents redundant token consumption across multi-file operations.
Ambient Activation	Injected unconditionally into the agent's context whenever working in that directory subtree.	Guarantees strict compliance without relying on model-triggered matching.

2. Implementing Safety & Security Guardrails

Safety guardrails protect the development workstation, production infrastructure, and sensitive credentials:

- **Credential Redaction:** Strict rules against outputting or committing `.env` files, API keys, and bearer tokens.
- **Destructive Command Prohibition:** Forbidding recursive force removals (`rm -rf`), hard git resets, or unbacked database drops.
- **Workspace Boundary Isolation:** Confining all tool modifications strictly to project workspace directories.

3. Implementing Code Formatting & Quality Standards

Formatting rules eliminate code style drift and maintain codebase uniformity across human and AI contributions:

- **Language Rules:** Enforce Prettier / ESLint for TypeScript, Black / Ruff for Python, and Google Style Guide for Go/C++.
- **Type Safety:** Disallow untyped `any` in TypeScript and mandate type hints on public Python function interfaces.
- **Comment & Docstring Integrity:** Mandate preservation of existing documentation during refactoring tasks.

4. Production .agents/AGENTS.md File Implementation

Below is the complete, battle-tested rule implementation authored in `.agents/AGENTS.md`:

```
# Project Operational Rules & Safety Protocols

## 1. Safety & Security Guardrails
- Credential Protection: Never log, display, or hardcode API keys, secrets, or .env files.
- Destructive Operations: Never execute rm -rf, git reset --hard, or DROP TABLE without explicit user review.
- Workspace Boundary: Confine file edits strictly to the workspace root.

## 2. Code Formatting & Quality Standards
- TypeScript: Adhere to strict typing (no any), ESLint/Prettier, and async/await syntax.
- Python: Follow PEP 8 with Black/Ruff formatting and explicit type annotations.
- Docstrings: Preserve all existing comments; add JSDoc/docstrings on new exported symbols.

## 3. Automated Verification Mandate
- Run project test suites (npm test / pytest) and linters before completing tasks.

## 4. Deliverable Format Policy
- All explanations and formal reports must be compiled into professional PDF format via ReportLab.
```

5. Subdirectory Hierarchy & Scoped Overrides

For monorepos or multi-tier projects, you can place localized rules inside subdirectories. The agent will inherit the root rules and layer the subsystem rules on top:

```
/
├── .agents/
│   ├── AGENTS.md           # Global repository rules (Safety, general style)
│   └── frontend/
│       ├── AGENTS.md       # Frontend rules (React hooks, Tailwind CSS, Jest)
│       └── backend/
│           └── AGENTS.md    # Backend rules (FastAPI, SQLAlchemy, Pytest, SQL safety)
```

Deployment Status: The rule file `.agents/AGENTS.md` has been created in your repository. It is actively enforced across all ongoing and future agent turns.